

# ***Structure of Project and State Management***

By Prasertsak U., Ph.D

# Outlines

- Structure of Project
- Business Logic
- โค้ดอย่างง่ายที่ไม่ได้ใช้ State Management
- ตัวอย่างการใช้งาน State Management ด้วย *Provider* Package
- Exercise

# Structure of Project สำคัญอย่างไร?

- การวางโครงสร้างโปรเจกต์ (Project Structure) เปรียบเสมือนการวาง **"ผังเมือง"** ก่อนที่จะสร้างสิ่งปลูกสร้าง ถ้าวางผังไม่ดี การขยายตัวในอนาคตจะทำได้ยาก แต่ถ้าผังเมืองดี ทุกอย่างจะชัดเจน ดูแลง่าย ซ่อมแซมแก้ไข รวมถึงการขยายจะทำได้ง่าย
- Debugging & Testing – การแก้ไขปัญหาเมื่อเกิดข้อผิดพลาดทำได้รวดเร็ว
- การทำงานร่วมกันเป็นทีม (Maintainability & Collaboration) - การวางโครงสร้างที่ชัดเจนเป็นส่วน จะช่วยลดระยะเวลาการอธิบายภายในทีมได้อย่างมาก เพราะภายในทีมจะเข้าใจตรงกัน และยังสามารถลดโอกาสในการเกิด conflict ในการแก้ไขโค้ดได้มากอีกด้วย (Git Conflict)

# Structure of Project – โครงสร้างตัวอย่าง

- ตัวอย่างการจัดวางโครงสร้างสำหรับโปรเจกต์ขนาดเล็กถึงกลาง เพื่อให้ง่ายต่อการทำงาน เช่น

```
lib/  
├ main.dart      # ไฟล์หลักสำหรับเริ่มการทำงาน  
├ models/        # โฟลเดอร์เก็บ Class ที่เป็นโครงสร้างของข้อมูล (เช่น User, Product)  
├ pages/         # โฟลเดอร์เก็บ UI แยกเป็นหน้าต่าง ๆ  
├ services/      # โฟลเดอร์เก็บโค้ดที่ใช้ติดต่อกับ Hardware/Database (เช่น LocalStorageService)  
├ providers/     # (หรือ controllers) โฟลเดอร์เก็บ Business Logic ต่าง ๆ  
└ widgets/       # โฟลเดอร์เก็บ UI ขนาดเล็ก ที่มีการใช้ซ้ำในหน้า Pages ต่าง ๆ
```

# Business Logic หมายถึงอะไร?

- Business Logic คือชุดของกฎและกระบวนการที่กำหนดวิธีการทำงาน เพื่อให้การตัดสินใจและการทำงานเป็นไปอย่างมีเหตุผล และสอดคล้องกับเป้าหมาย
- ตัวอย่าง เช่น
  - ระบบลงทะเบียนเรียน: ถ้านักศึกษาลงวิชาใหม่ → ระบบต้องตรวจสอบว่ามีที่นั่งว่างหรือไม่ → ถ้าเต็มแล้วต้องแจ้งเตือน
  - แอปธนาคาร: ถ้าผู้ใช้โอนเงินเกินวงเงินที่กำหนด → ระบบต้องปฏิเสธและแจ้งเตือน

## ระบบธนาคาร

- UI: ผู้ใช้กรอกจำนวนเงินที่จะโอน
- Business Logic: ตรวจสอบยอดเงินคงเหลือ, ตรวจสอบวงเงินสูงสุดต่อวัน, ถ้าเกินวงเงินให้ปฏิเสธ
- Database: บันทึกธุรกรรมการโอน

# เนื้อหาจากคาบเรียนที่ผ่านมา (ต่อเนื่อง)

- ให้สร้างโปรเจกต์ใหม่ชื่อ **bodyhealth\_app**
- สร้าง folder ชื่อว่า **model** และ **pages** ภายใน **/lib**
- สร้างไฟล์ชื่อ **bmr\_pages.dart** ไว้ภายใน **pages**
- สร้างไฟล์ชื่อ **body\_profile.dart** ไว้ภายใน **models** มีโครงสร้างประกอบด้วย
  - weight - **double**
  - height - **double**
  - age - **int**
  - gender - **String**

## โครงสร้างไฟล์

```
/lib
- main.dart
- model/body_profile.dart
- pages/bmr_page.dart
```

## รายละเอียดภายในไฟล์ body\_profile.dart

```
1 class BodyProfile {
2   · String gender;
3   · int age;
4   · double weight;
5   · double height;
6
7   · BodyProfile({
8     · required this.gender,
9     · required this.age,
10    · required this.weight,
11    · required this.height,
12  · });
13 }
```

# เนื้อหาจากคาบเรียนที่ผ่านมา (ต่อเนื่อง)

- สร้างแบบฟอร์มสำหรับกรอกข้อมูล ในไฟล์ **bmr\_page.dart** ภายในโฟลเดอร์ **pages** เพื่อนำไปใช้ในการคำนวณ ค่า BMR โดยแบบฟอร์มประกอบด้วย น้ำหนัก ส่วนสูง อายุ เพศ (ชาย/หญิง) และมี 2 ปุ่มสำหรับกดเพื่อเริ่มคำนวณ และการ reset ค่า
- นำข้อมูลที่กรอกเก็บลงในตัวแปรภายในคลาส **BodyProfile** ที่สร้างไว้
- ลักษณะหน้าจอที่ต้องการ ให้แสดงมีดังนี้

\* ให้ทำการ Validate ความถูกต้องของข้อมูลให้ครบถ้วน

The image shows a mobile application interface for a BMR Calculator. The title bar is purple with the text 'BMR Calculator' in white. Below the title bar, there are four input sections: 1. Gender: A purple icon of two people is on the left. To the right are two radio buttons, 'Male' (selected) and 'Female'. 2. Weight (kg): A purple icon of a scale is on the left. To the right is a text input field containing '65'. 3. Height (cm): A purple icon of a person is on the left. To the right is a text input field containing '170'. 4. Age (years): A purple icon of a person is on the left. To the right is a horizontal slider ranging from 0 to 100, with a purple circle indicating the current value around 25. At the bottom of the form are two rounded rectangular buttons: 'Calculate' and 'Reset'.

\* ยังไม่ต้องทำการคำนวณ เพียงสร้างฟอร์มกรอกข้อมูลเท่านั้น

# ตัวอย่าง APP

- จากโปรเจกต์ **bodyhealth\_app** เดิมที่ได้จากเนื้อหาที่ผ่านมา
- สำหรับคนที่ไม่ได้สร้าง `bodyhealth_app` หรือไม่ได้ทำมาก่อนให้ดาวน์โหลดจาก [ที่นี่](#)
- ให้ปรับปรุงโครงสร้างโปรเจกต์ด้วยการสร้างโฟลเดอร์เพิ่มเติมดังนี้
- สร้างโฟลเดอร์ภายใน **/lib** ให้ประกอบด้วยโฟลเดอร์ต่อไปนี้

```
lib/  
└─ main.dart      # ไฟล์หลักสำหรับเริ่มการทำงาน (เดิม-มีอยู่แล้ว)  
└─ models/        # โฟลเดอร์เก็บ Class ที่เป็นโครงสร้างของข้อมูล (เดิม-มีอยู่แล้ว)  
└─ pages/         # โฟลเดอร์เก็บ UI แยกเป็นหน้าต่าง ๆ (เดิม-มีอยู่แล้ว)  
└─ services/      # โฟลเดอร์เก็บโค้ดที่ใช้ติดต่อกับ Hardware/Database (สร้างเพิ่มเติม)  
└─ providers/     # (หรือ controllers) โฟลเดอร์เก็บ Business Logic ต่าง ๆ (สร้างเพิ่มเติม)
```



# สร้าง Code เพื่อรองรับ Logic ในการคำนวณ BMR

- สร้างไฟล์ชื่อ *bmr\_provider.dart* ไว้ในโฟลเดอร์ **providers**
- เขียนโค้ดเพื่อคำนวณค่า BMR จากสูตรต่อไปนี้

ผู้ชาย:  $BMR = 10W + 6.25H - 5A + 5$

ผู้หญิง:  $BMR = 10W + 6.25H - 5A - 161$

## หมายเหตุ

W = น้ำหนัก หน่วยเป็น กิโลกรัม

H = ความสูง หน่วยเป็น เซนติเมตร

A = อายุ หน่วยเป็น ปี

# สร้าง Code เพื่อรองรับ Logic ในการคำนวณ BMR

bmr\_provider.dart

```
class BmrProvider {  
  static double calculateBMR(  
    double weight,  
    double height,  
    int age,  
    String gender,  
  ) {  
    if (gender == 'Male') {  
      return (10 * weight) + (6.25 * height) - (5 * age) + 5;  
    } else if (gender == 'Female') {  
      return (10 * weight) + (6.25 * height) - (5 * age) - 161;  
    }  
    throw ArgumentError(  
      'Invalid gender. Please specify either "Male" or "Female".',  
    );  
  }  
}
```

# สร้าง page สำหรับแสดงผลการคำนวณ

- สร้างไฟล์ชื่อ **bmr\_result\_page.dart** ไว้ในโฟลเดอร์ **pages**
- สำหรับ page ที่ใช้แสดงผลลัพธ์นี้ เมื่อเรียกใช้จะรับพารามิเตอร์ 2 ตัว ได้แก่ ค่า **bmr** ที่คำนวณได้ และ **object ของคลาส BodyProfile** ที่เก็บรายละเอียดต่างๆ ที่ใช้ในการคำนวณค่า bmr
- ผู้ที่จะเรียกใช้หน้าผลลัพธ์นี้จะต้องส่งข้อมูล 2 ตัวข้างต้นมาให้ด้วยเสมอ

# สร้าง page สำหรับแสดงผลการคำนวณ

bmr\_result\_page.dart

```
import 'package:flutter/material.dart';  
import 'package:bmr_app/models/body_profile.dart';
```

```
class BmrResultPage extends StatelessWidget {  
  final BodyProfile bodyProfile;  
  final double bmr;
```

```
  const BmrResultPage({  
    super.key,  
    required this.bodyProfile,  
    required this.bmr,  
  });
```

```
  @override  
  Widget build(BuildContext context) {  
    return Scaffold() // เนื้อหาส่วนนี้ต่อหน้าถัดไป  
  }  
}
```



จะต้องประกาศตัวแปร  
ที่ใช้เป็นตัวรับค่าจาก  
พารามิเตอร์ ตรงจุดนี้



ส่วนของการกำหนด  
พารามิเตอร์ที่ต้องการ



ส่วนภายใน scaffold  
ต่อหน้าถัดไป

# สร้าง page สำหรับแสดงผลการคำนวณ

bmr\_result\_page.dart (ต่อ)

```
return Scaffold(  
  appBar: AppBar(  
    centerTitle: true,  
    backgroundColor: Colors.deepPurple,  
    foregroundColor: Colors.white,  
    title: Text('BMR Result'),  
  ),  
  body: Center(  
    child: Container(  
      margin: EdgeInsets.all(16),  
      child: Column(  
        children: [  
          Text('Your BMR:', style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),  
          SizedBox(height: 16),  
          Text('${bmr.toStringAsFixed(2)} calories/day', style: TextStyle(fontSize: 16)),  
          SizedBox(height: 16),  
          Text('Details:', style: TextStyle(fontSize: 18, fontWeight: FontWeight.bold)),  
          SizedBox(height: 8),  
          Text('Gender: ${bodyProfile.gender}', style: TextStyle(fontSize: 16)),  
          Text('Age: ${bodyProfile.age}', style: TextStyle(fontSize: 16)),  
          Text('Weight: ${bodyProfile.weight.toStringAsFixed(2)} kg', style: TextStyle(fontSize: 16)),  
          Text('Height: ${bodyProfile.height.toStringAsFixed(2)} cm', style: TextStyle(fontSize: 16)),  
        ],  
      ),  
    ),  
  ),  
);
```

# เขียนโค้ดสำหรับเรียกใช้ Logic การคำนวณ

- แก้ไขไฟล์ชื่อ `bmr_page.dart` ในโฟลเดอร์ `pages`
- การแก้ไขประกอบด้วย
  - การสร้างเมธอดหรือฟังก์ชันสำหรับเรียกใช้เมธอดการคำนวณใน providers
  - การตรวจสอบความถูกต้องของข้อมูลภายใน Form ก่อนการคำนวณ เนื่องจากผู้ใช้อาจกดปุ่มคำนวณทันทีโดยข้อมูลยังไม่ครบถ้วน
  - การเพิ่มโค้ดสำหรับการ reset ค่าภายใน Form
  - การเขียนโค้ดเพื่อแสดงผลลัพธ์ หลังจากการคำนวณค่า BMR เสร็จเรียบร้อยแล้ว โดยจะต้องส่งค่า BMR ที่คำนวณได้ พร้อมกับรายละเอียดของ `body_profile`

# เขียนโค้ดสำหรับเรียกใช้ Logic การคำนวณ

- เพิ่มเมธอดสำหรับการคำนวณ โดยสร้างเมธอดชื่อ **doCalculation()**

```
void doCalculation() {  
    try {  
        double bmr = BmrProvider.calculateBMR(  
            bodyProfile.weight,  
            bodyProfile.height,  
            bodyProfile.age,  
            bodyProfile.gender,  
        );  
        // show result in a new page  
        Navigator.push(  
            context,  
            MaterialPageRoute(  
                builder: (context) =>  
                    BmrResultPage(bodyProfile: bodyProfile, bmr: bmr),  
            ),  
        );  
    } catch (e) {  
        ScaffoldMessenger.of(context).showSnackBar(  
            SnackBar(  
                content: Text('Error calculating BMR: ${e.toString()}'),  
                backgroundColor: Colors.red,  
            ),  
        );  
    }  
}
```



เรียกใช้ logic การคำนวณใน providers



เปิดหน้า result เพื่อแสดงผล



แสดงแถบข้อความกรณีเกิดข้อผิดพลาด

# เขียนโค้ดสำหรับเรียกใช้ Logic การคำนวณ

- เพิ่มโค้ดสำหรับตรวจสอบความถูกต้องของข้อมูลก่อนการคำนวณ ภายใต้ปุ่ม *Calculate*

```
Expanded(  
  child: OutlinedButton(  
    onPressed: () {  
      // Handle form submission  
      if (_formKey.currentState!.validate()) {  
        doCalculation();  
      } else {  
        ScaffoldMessenger.of(context).showSnackBar(  
          SnackBar(  
            content: Text(  
              'Please correct the errors in the form.',  
            ), // Text  
            backgroundColor: Colors.red,  
          ), // SnackBar  
        );  
      }  
    },  
    child: Text('Calculate'),  
  ), // OutlinedButton  
, // Expanded
```

ใช้คำสั่ง if เพื่อตรวจสอบความถูกต้องของข้อมูลทั้งหมดภายใน form หากมีข้อมูลใดที่ยังไม่ถูกต้องตามเงื่อนไขการ validate จะได้ค่าเป็น **false** กลับมา ซึ่งจะทำงานในส่วนของ else โดยจะแสดงแถบข้อความด้านล่างจอตด้วยพื้นหลังสีแดง แต่ถ้าข้อมูลถูกต้องจะได้ค่า **true** และจะเรียกใช้ **doCalculation()**





# เขียนโค้ดสำหรับเรียกใช้ Logic การคำนวณ

- เพิ่มโค้ดสำหรับการล้างค่าหรือ reset ค่าภายใน Form ภายใต้อุปกรณ์ *Reset*

```
Expanded(  
  child: OutlinedButton(  
    onPressed: () {  
      // Handle form reset  
      _formKey.currentState?.reset();  
    },  
    child: Text('Reset'),  
  ), // OutlinedButton  
) // Expanded
```

คำสั่งสำหรับล้างค่าหรือ reset ค่าทั้งหมด  
ภายใน Form



# ทดสอบการทำงาน

BMR Calculator

☐ Male

☒ Female

Weight (kg)

50

Height (cm)

Height is required.

Age:(years)

025100

Calculate

Reset

Please correct the errors in the form.

BMR Calculator

☐ Male

☒ Female

Weight (kg)

50

Height (cm)

155

Age:(years)

021100

Calculate

Reset

BMR Result

Your BMR:

1202.75 calories/day

Details:

Gender: Female

Age: 21

Weight: 50.00 kg

Height: 155.00 cm

# ปัญหาชวนคิด..?!

- หากมีความต้องการใช้งานค่า BMR ในหน้าอื่นๆ อีกมากกว่า 1 จุด จะทำอย่างไร..?

จะต้องส่งข้อมูล BodyProfile ไปคำนวณใหม่ หรือจำค่าเดิมไว้ แล้วส่งไปแสดงยังจุดอื่น ๆ ดีหรือไม่ ?

การสร้าง Object ขึ้นมาใหม่ทุกครั้งที่ต้องการคำนวณ เช่น ถ้าสร้าง object ใน Page A เพื่อคำนวณ และต้องสร้างอีกตัวใน Page B ข้อมูลที่คำนวณไว้ใน Page A จะไม่ตามไปที่ Page B เพราะเป็นคนละก้อนกัน การสร้างใหม่เรื่อยๆ ทำให้แอปทำงานหนักขึ้นโดยไม่จำเป็น และเป็นการสิ้นเปลืองหน่วยความจำ

- หากต้องมีการแสดงค่า BMR ในเวลาเดียวกัน หลายจุดในหน้า Page จะต้องทำอย่างไรให้ค่าที่แสดงในแต่ละจุดอัปเดตเป็นค่าปัจจุบันอยู่เสมอ การใช้ setState() ถ้า App ขนาดเล็กก็ใช้ได้ แต่ถ้า App มีขนาดใหญ่ขึ้น การใช้ setState() ไปทุกแห่งที่มีการแสดงผลเพื่ออัปเดต UI นั้นค่อนข้างยุ่งยาก ไม่สวยงาม และทำให้โค้ดซับซ้อน แก้ไขได้ยากในอนาคต

# แนวทางการแก้ไข...!

- ใช้คนกลางในการจัดการ State โดยยึดหลัก เมื่อ State เปลี่ยน → UI ต้องเปลี่ยนตาม
- วิธีการจัดการ State และการกระจาย State ไปยัง Widget ต่างๆ ทำอย่างไร ในที่นี้จะใช้ **provider package** เป็นตัวช่วย โดยจะสามารถเข้าถึงข้อมูลจากที่ไหนก็ได้
- widget ต่างๆ สามารถคอยฟัง Notify ได้ว่า state ของข้อมูลที่สนใจนั้น มีการเปลี่ยนแปลงหรือไม่ และสามารถทำการ Rebuild UI ได้ทันที หากข้อมูลที่กำลังสนใจนั้นมีการเปลี่ยนแปลง

# ลักษณะการเฝ้าฟัง Notify จาก Provider

- ลักษณะการคอยฟัง Notify ของ Provider แบ่งได้ 2 ลักษณะ ดังนี้

| คำสั่ง                                | การใช้งาน                                         | ใช้ตอนไหน                                     |
|---------------------------------------|---------------------------------------------------|-----------------------------------------------|
| <code>context.read&lt;T&gt;()</code>  | เข้าไปสั่งงาน หรือดึงค่า "ครั้งเดียว"             | มักใช้ใน ฟังก์ชัน เช่น <code>onPressed</code> |
| <code>context.watch&lt;T&gt;()</code> | เฝ้าดูข้อมูล ถ้าข้อมูลเปลี่ยน<br>ให้วาดหน้าจอใหม่ | มักใช้ใน <code>build()</code> ของ<br>Widget   |

**T - Generic Type Parameter** ในที่นี่จะถูกแทนที่ด้วยชื่อคลาสที่ต้องการใช้งาน

# แก้ไขโค้ด `bodyhealth_app` ให้ใช้งาน `State Management`

- นำเข้า `provider package` มาใช้ในโปรเจกต์ ด้วยการเพิ่มเติมชื่อ `provider package` ไว้ในไฟล์ `pubspec.yaml`

`pubspec.yaml`

```
30 dependencies:
31   flutter:
32     sdk: flutter
33
34   # The following adds the Cupertino Icons font to your application.
35   # Use with the CupertinoIcons class for iOS style icons.
36   cupertino_icons: ^1.0.8
37   flutter_form_builder: ^10.3.0
38   form_builder_validators: ^11.3.0
39   provider: ^6.1.5
40
```

## แก้ไขโค้ด `bodyhealth_app` ให้ใช้งาน **State Management**

- สร้างไฟล์ชื่อ `bmr_service.dart` ไว้ในโฟลเดอร์ `/lib/services`
- เขียนโค้ดสำหรับคำนวณค่า BMR ไว้ในส่วนนี้แทน

`bmr_service.dart`

```
class BmrService {  
  double calculateBMR(double weight, double height, int age, String gender) {  
    if (gender == 'Male') {  
      return (10 * weight) + (6.25 * height) - (5 * age) + 5;  
    } else if (gender == 'Female') {  
      return (10 * weight) + (6.25 * height) - (5 * age) - 161;  
    }  
    throw ArgumentError(  
      'Invalid gender. Please specify either "Male" or "Female".',  
    );  
  }  
}
```

# แก้ไขโค้ด `bodyhealth_app` ให้ใช้งาน `State Management`

- แก้ไขไฟล์ `bmr_provider.dart` ให้เรียกใช้การคำนวณค่า bmr จาก service แทนการคำนวณด้วยตนเอง โดยย้ายให้ตัวเองเป็นเพียงคนส่งสาร

```
import 'package:flutter/material.dart';
import '../models/body_profile.dart';
import '../services/bmr_service.dart';
class BmrProvider extends ChangeNotifier {
  double? _bmrResult;
  final BmrService _bmrService = BmrService();

  double getLastResult() {
    return _bmrResult ?? 0.0; // ถ้า _bmrResult ยังเป็น null ให้คืนค่า 0.0 แทน
  }

  void processBmrCalculation(BodyProfile profile) {
    try {
      // 1. ส่งสารไปให้ Service คำนวณ
      _bmrResult = _bmrService.calculateBMR(
        profile.weight,
        profile.height,
        profile.age,
        profile.gender,
      );
      notifyListeners(); // แจ้งคนอื่นว่าคำนวณเสร็จแล้ว
    } catch (e) {
      rethrow; // ส่ง Error กลับไปให้ UI ที่เป็นคนเรียกใช้เพื่อแสดงข้อความผิดพลาด
    }
  }
}
```



# แก้ไขโค้ด `bodyhealth_app` ให้ใช้งาน **State Management**

- แก้ไขไฟล์ `bmr_page.dart` ในส่วนของเมธอด `doCalculation()`

ให้เพิ่มการ import provider ไว้ในส่วนหัวด้วย: `import 'package:provider/provider.dart';`

```
void doCalculation() {  
  try {  
    // ส่งผ่าน Provider (คนส่งสาร)  
    context.read<BmrProvider>().processBmrCalculation(bodyProfile);  
  
    // ดึงค่าผลลัพธ์มาแสดง  
    double bmr = context.read<BmrProvider>().getLastResult();  
  
    Navigator.push(  
      context,  
      MaterialPageRoute(  
        builder: (context) =>  
          BmrResultPage(bodyProfile: bodyProfile, bmr: bmr),  
      ),  
    );  
  } catch (e) {  
    ScaffoldMessenger.of(context).showSnackBar(  
      SnackBar(  
        content: Text('Error calculating BMR: ${e.toString()}'),  
        backgroundColor: Colors.red,  
      ),  
    );  
  }  
}
```

# แก้ไขโค้ด `bodyhealth_app` ให้ใช้งาน **State Management**

- แก้ไขไฟล์ `main.dart` ด้วยการประกาศส่วนของ Provider ที่จะใช้งาน

ให้เพิ่มการ import provider ไว้ในส่วนหัวด้วย: `import 'package:provider/provider.dart';`

```
void main() {  
  runApp(  
    // ใช้ MultiProvider เพื่อรองรับหลาย Provider ในอนาคต  
    MultiProvider(  
      providers: [ChangeNotifierProvider(create: (_) => BmrProvider())],  
      child: const MyApp(),  
    ),  
  );  
}
```

# หมายเหตุ

- คำสั่ง ***notifyListeners()***  
หากไม่มี ก็ไม่ทำให้โค้ดผิด  
เพียงแต่ widget ที่มีการเฝ้า  
ฟังสถานะของข้อมูล bmr  
จะไม่ได้รับ notify ใด ๆ  
เมื่อค่า bmr  
เกิดการเปลี่ยนแปลง  
จะส่งผลให้การแสดงผล  
ไม่อัปเดตตามความเป็นจริง

```
5 class BmrProvider extends ChangeNotifier {
6   double? _bmrResult;
7   final BmrService _bmrService = BmrService();
8
9   double getLastResult() {
10    return _bmrResult ?? 0.0; // ถ้า _bmrResult ยังเป็น null ให้คืนค่า 0.0 แทน
11  }
12
13  void processBmrCalculation(BodyProfile profile) {
14    try {
15      // 1. ส่งสารไปให้ Service คำนวณ
16      _bmrResult = _bmrService.calculateBMR(
17        profile.weight,
18        profile.height,
19        profile.age,
20        profile.gender,
21      );
22
23      notifyListeners(); // แจ้งคนอื่นว่าคำนวณเสร็จแล้ว
24    } catch (e) {
25      rethrow; // ส่ง Error กลับไปให้ UI ที่เป็นคนเรียกใช้เพื่อแสดงข้อความผิดพลาด
26    }
27  }
28 }
```

# การบ้าน #4

\* ให้ใช้ provider package

- เพื่อให้เกิดความเข้าใจเกี่ยวกับ State Management โดยใช้ provider ให้นิสิตสร้าง App ชื่อว่า **test\_provider** โดยมีรายละเอียด ดังนี้

- ให้มีหน้าหลัก **home\_page.dart** อยู่ภายในโฟลเดอร์ **/lib/pages** ภายในหน้า home นี้ให้มีข้อความขนาด 1 ย่อหน้า (ให้ปริมาณคำเกือบเต็มหน้าหรือเกินกว่านั้น โดยสร้างเนื้อหาขึ้นเองอะไรก็ได้)
- ภายในหน้า home ให้มี icon บน AppBar สำหรับเปิดหน้าจอ Setting ขึ้นมา
- หน้าจอ Setting ให้สร้างไว้ใน **/lib/pages** โดยใช้ชื่อ **setting\_page.dart** การ setting ให้ผู้ใช้สามารถเลือกปรับได้ 2 อย่างได้แก่
  - เลือกโหมดแสดงผลระหว่าง Dark Mode และ Light Mode
  - เลือกปรับขนาดตัวอักษรได้ 3 ขนาดคือ Small, Medium และ Large
- โดยในตอนเริ่มโปรแกรมให้กำหนดโหมดเป็น Light Mode ขนาดตัวอักษรเป็น Medium
- เมื่อออกจากหน้าจอ Setting กลับมายังหน้า Home แล้ว สิ่งที่จะเลือกจะต้องเห็นผลทันทีโดยอัตโนมัติ

\* ส่งในคาบเรียนครั้งถัดไป

ขนาดตัวอักษร Small, Medium และ Large ให้นิสิตกำหนด Font Size เองว่าในแต่ละขนาด จะใช้ Size ขนาดเป็นตัวเลขเท่าใด

Q/A